

PySpark UDF (a.k.a User Defined Function) is the most useful feature of Spark SQL & DataFrame that is used to extend the PySpark build in capabilities. Note: UDF's are the most expensive operations hence use them only you have no choice and when essential.

1. PySpark UDF Introduction

1.1 What is UDF?

UDF's a.k.a User Defined Functions, If you are coming from SQL background, UDF's are nothing new to you as most of the traditional RDBMS databases support User Defined Functions, these functions need to register in the database library and use them on SQL as regular functions. PySpark UDF's are similar to UDF on traditional databases. In PySpark, you create a function in a Python syntax and wrap it with PySpark SQL `udf()` or register it as `udf` and use it on DataFrame and SQL respectively.

1.2 Why do we need a UDF?

UDF's are used to extend the functions of the framework and re-use these functions on multiple DataFrame's. For example, you wanted to convert every first letter of a word in a name string to a capital case; PySpark build-in features don't have this function hence you can create it a UDF and reuse this as needed on many Data Frames. UDF's are once created they can be re-used on several DataFrame's and SQL expressions. Before you create any UDF, do your research to check if the similar function you wanted is already available in Spark SQL Functions. PySpark SQL provides several predefined common functions and many more new functions are added with every release. hence, It is best to check before you reinventing the wheel. When you creating UDF's you need to design them very carefully otherwise you will come across optimization & performance issues.

2. Create PySpark UDF

2.1 Create a DataFrame

Before we jump in creating a UDF, first let's create a PySpark DataFrame.

```
In [1]: import findspark
findspark.init()
```

```
In [2]: from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

columns = ["Seqno", "Name"]

data = [("1", "John Jones"),
        ("2", "Tracey Smith"),
        ("3", "Amy Sanders")]

df = spark.createDataFrame(data = data, schema = columns)

df.show(truncate = False)
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones|
|2     |Tracey Smith|
|3     |Amy Sanders|
+-----+-----+
```

2.2 Create a Python Function

The first step in creating a UDF is creating a Python function. Below snippet creates a function `convertCase()` which takes a string parameter and converts the first letter of every word to capital letter. UDF's take parameters of your choice and returns a value.

```
In [3]: def convertCase(str):
        resStr = ""
        arr = str.split(" ")
        for x in arr:
            resStr = resStr + x[0:1].upper() + x[1:len(x)] + " "
        return resStr
```

Note that there might be a better way to write this function. But for the sake of this article, I am not worried much about the performance and better ways.

2.3 Convert a Python function to PySpark UDF

Now convert this function `convertCase()` to UDF by passing the function to PySpark SQL `udf()`, this function is available at `org.apache.spark.sql.functions.udf` package. Make sure you import this package before using it. PySpark SQL `udf()` function returns `org.apache.spark.sql.expressions.UserDefinedFunction` class object.

```
In [4]: from pyspark.sql.functions import col,udf
        from pyspark.sql.types import StringType

        # Converting function to UDF
        convertUDF = udf(lambda z: convertCase(z), StringType())
```

Note: The default type of the udf() is StringType hence, you can also write the above statement without return type.

```
In [5]: # Converting function to UDF
        # StringType() is by default hence not required
        convertUDF = udf(lambda z: convertCase(z))
```

3. Using UDF with DataFrame

3.1 Using UDF with PySpark DataFrame select()

Now you can use convertUDF() on a DataFrame column as a regular build-in function.

```
In [6]: df.select(col("Seqno"), \
                convertUDF(col("Name")).alias("Name") ) \
        .show(truncate = False)
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones|
|2     |Tracey Smith|
|3     |Amy Sanders|
+-----+-----+
```

3.2 Using UDF with PySpark DataFrame withColumn()

You could also use udf on DataFrame withColumn() function, to explain this I will create another upperCase() function which converts the input string to upper case.

```
In [7]: def upperCase(str):
        return str.upper()
```

Let's convert upperCase() python function to UDF and then use it with DataFrame withColumn(). Below example converts the values of "Name" column to upper case and creates a new column "Curated Name"

```
In [8]: upperCaseUDF = udf(lambda z:upperCase(z), StringType())

        df.withColumn("Curated Name", upperCaseUDF(col("Name"))) \
        .show(truncate = False)
```

```
+-----+-----+-----+
|Seqno|Name      |Curated Name|
+-----+-----+-----+
|1     |John Jones|JOHN JONES  |
|2     |Tracey Smith|TRACEY SMITH|
|3     |Amy Sanders|AMY SANDERS |
+-----+-----+-----+
```

3.3 Registering PySpark UDF & use it on SQL

In order to use convertCase() function on PySpark SQL, you need to register the function with PySpark by using spark.udf.register().

```
In [9]: """ Using UDF on SQL """
        spark.udf.register("convertUDF", convertCase, StringType())
        df.createOrReplaceTempView("NAME_TABLE")
        spark.sql("SELECT seqno, convertUDF(Name) AS name FROM NAME_TABLE") \
        .show(truncate = False)
```

```
+-----+-----+
|seqno|name      |
+-----+-----+
|1     |John Jones|
|2     |Tracey Smith|
|3     |Amy Sanders|
+-----+-----+
```

This yields the same output as 3.1 example.

4. Creating UDF using annotation

In the previous sections, you have learned creating a UDF is a 2 step process, first, you need to create a Python function, second convert

function to UDF using SQL udf() function, however, you can avoid these two steps and create it with just a single step by using annotations.

```
In [10]: @udf(returnType=StringType())
def upperCase(str):
    return str.upper()

df.withColumn("Curated Name", upperCase(col("Name"))) \
    .show(truncate = False)
```

	Seqno	Name	Curated Name
	1	John Jones	JOHN JONES
	2	Tracey Smith	TRACEY SMITH
	3	Amy Sanders	AMY SANDERS

This results same output as section 3.2

5. Special Handling

5.1 Execution order

One thing to aware is in PySpark/Spark does not guarantee the order of evaluation of subexpressions meaning expressions are not guarantee to evaluated left-to-right or in any other fixed order. PySpark reorders the execution for query optimization and planning hence, AND, OR, WHERE and HAVING expression will have side effects. So when you are designing and using UDF, you have to be very careful especially with null handling as these results runtime exceptions.

```
In [11]: """
No guarantee Name is not null will execute first
If convertUDF(Name) like '%John%' execute first then
you will get runtime error
"""

spark.sql("SELECT Seqno, convertUDF(Name) AS Name FROM NAME_TABLE" + \
    "where Name is not null and convertUDF(Name) like '%John%'" ) \
    .show(truncate = False)
```

```

-----
ParseException                                Traceback (most recent call last)
Cell In[11], line 6
      1 """
      2 No guarantee Name is not null will execute first
      3 If convertUDF(Name) like '%John%' execute first then
      4 you will get runtime error
      5 """
----> 6 spark.sql("SELECT Seqno, convertUDF(Name) AS Name FROM NAME_TABLE" + \
      7           "where Name is not null and convertUDF(Name) like '%John%'" ) \
      8           .show(truncate = False)

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\pyspark\sql\session.py:1631, in SparkSession.sql(self, sqlQuery, args, **kwargs)
    1627         assert self._jvm is not None
    1628         litArgs = self._jvm.PythonUtils.toArray(
    1629             [_to_java_column(lit(v)) for v in (args or [])]
    1630         )
-> 1631         return DataFrame(self._jsparkSession.sql(sqlQuery, litArgs), self)
    1632     finally:
    1633         if len(kwargs) > 0:

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\lib\py4j-0.10.9.7-src.zip\py4j\java_gateway.py:1322, in JavaMember._call_(self, *args)
    1316 command = proto.CALL_COMMAND_NAME + \
    1317             self.command_header + \
    1318             args_command + \
    1319             proto.END_COMMAND_PART
    1321 answer = self.gateway_client.send_command(command)
-> 1322 return_value = get_return_value(
    1323     answer, self.gateway_client, self.target_id, self.name)
    1325 for temp_arg in temp_args:
    1326     if hasattr(temp_arg, "_detach"):

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\pyspark\errors\exceptions\captured.py:185, in capture_sql_exception.<locals>.<decorator>(*a, **kw)
    181 converted = convert_exception(e.java_exception)
    182 if not isinstance(converted, UnknownException):
    183     # Hide where the exception came from that shows a non-Pythonic
    184     # JVM exception message.
--> 185     raise converted from None
    186 else:
    187     raise

ParseException:
[PARSE_SYNTAX_ERROR] Syntax error at or near 'is'.(line 1, pos 65)

== SQL ==
SELECT Seqno, convertUDF(Name) AS Name FROM NAME_TABLEwhere Name is not null and convertUDF(Name) like '%John%'
-----

```

5.2 Handling null check

UDF's are error-prone when not designed carefully. for example, when you have a column that contains the value null on some records

```

In [12]: """ null check """

columns = ["Seqno", "Name"]
data = [
    ("1", "John Jones"),
    ("2", "Tracey Smith"),
    ("3", "Amy Sanders"),
    ("4", None)
]

df2 = spark.createDataFrame(data = data, schema = columns)
df2.show(truncate = False)
df2.createOrReplaceTempView("NAME_TABLE2")

spark.sql("SELECT convertUDF(Name) FROM NAME_TABLE2") \
    .show(truncate = False)

```

```

+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones|
|2     |Tracey Smith|
|3     |Amy Sanders|
|4     |NULL      |
+-----+-----+

```

PythonException

Traceback (most recent call last)

Cell In[12], line 15

```
11 df2.show(truncate = False)
12 df2.createOrReplaceTempView("NAME_TABLE2")
14 spark.sql("SELECT convertUDF(Name) FROM NAME_TABLE2") \
--> 15     .show(truncate = False)
```

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\pyspark\sql\dataframe.py:945, in DataFrame.show(self, n, truncate, vertical)

```
885 def show(self, n: int = 20, truncate: Union[bool, int] = True, vertical: bool = False) -> None:
886     """Prints the first ``n`` rows to the console.
887
888     .. versionadded:: 1.3.0
889
890     (...)
943     name | Bob
944     """
--> 945     print(self._show_string(n, truncate, vertical))
```

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\pyspark\sql\dataframe.py:976, in DataFrame._show_string(self, n, truncate, vertical)

```
967 except ValueError:
968     raise PySparkTypeError(
969         error_class="NOT_BOOL",
970         message_parameters={
971             ...
973         },
974     )
--> 976 return self._jdf.showString(n, int_truncate, vertical)
```

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\lib\py4j-0.10.9.7-src.zip\py4j\java_gateway.py:1322, in JavaMember._call__(self, *args)

```
1316 command = proto.CALL_COMMAND_NAME + \
1317     self.command_header + \
1318     args_command + \
1319     proto.END_COMMAND_PART
1321 answer = self.gateway_client.send_command(command)
-> 1322 return_value = get_return_value(
1323     answer, self.gateway_client, self.target_id, self.name)
1325 for temp_arg in temp_args:
1326     if hasattr(temp_arg, "_detach"):
```

File C:\Program Files\spark-3.5.1-bin-hadoop3\python\pyspark\errors\exceptions\captured.py:185, in capture_sql_exception.<locals>.deco(*a, **kw)

```
181 converted = convert_exception(e.java_exception)
182 if not isinstance(converted, UnknownException):
183     # Hide where the exception came from that shows a non-Pythonic
184     # JVM exception message.
--> 185     raise converted from None
186 else:
187     raise
```

PythonException:

An exception was thrown from the Python worker. Please see the stack trace below.

Traceback (most recent call last):

File "C:\Users\Dell\AppData\Local\Temp\ipykernel_18024\595000715.py", line 3, in convertCase
AttributeError: 'NoneType' object has no attribute 'split'

Note that from the above snippet, record with “Seqno 4” has value “None” for “name” column. Since we are not handling null with UDF function, using this on DataFrame returns below error. Note that in Python None is considered null. Below points to remember Its always best practice to check for null inside a UDF function rather than checking for null outside. In any case, if you can’t do a null check in UDF at lease use IF or CASE WHEN to check for null and call UDF conditionally.

```
In [13]: spark.udf.register("_nullsafeUDF", lambda str: convertCase(str) if not str is None else "", StringType())

spark.sql("SELECT _nullsafeUDF(Name) FROM NAME_TABLE2") \
    .show(truncate = False)

spark.sql("SELECT Seqno, _nullsafeUDF(Name) AS Name FROM NAME_TABLE2 " + \
    " WHERE Name is not null and _nullsafeUDF(Name) like '%JOHN%'" ) \
    .show(truncate = False)
```

```

+-----+
|_nullsafeUDF(Name)|
+-----+
|John Jones        |
|Tracey Smith      |
|Amy Sanders       |
|                  |
+-----+

+-----+-----+
|Seqno|Name|
+-----+-----+
+-----+-----+

```

This executes successfully without errors as we are checking for null/none while registering UDF.

5.3 Performance concern using UDF

UDFs are a black box to PySpark hence it can't apply optimization and you will lose all the optimization PySpark does on Dataframe/Dataset. When possible you should use Spark SQL built-in functions as these functions provide optimization. Consider creating UDF only when the existing built-in SQL function doesn't have it.

6. Complete PySpark UDF Example

Below is a complete UDF function example in Python

```

In [15]: import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, udf
from pyspark.sql.types import StringType

spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()

columns = ["Seqno", "Name"]
data = [("1", "john jones"),
        ("2", "tracey smith"),
        ("3", "amy sanders")]

df = spark.createDataFrame(data=data, schema=columns)

df.show(truncate=False)

def convertCase(str):
    resStr=""
    arr = str.split(" ")
    for x in arr:
        resStr= resStr + x[0:1].upper() + x[1:len(x)] + " "
    return resStr

""" Converting function to UDF """
convertUDF = udf(lambda z: convertCase(z))

df.select(col("Seqno"), \
          convertUDF(col("Name")).alias("Name") ) \
.show(truncate=False)

def upperCase(str):
    return str.upper()

upperCaseUDF = udf(lambda z:upperCase(z),StringType())

df.withColumn("Cureated Name", upperCaseUDF(col("Name"))) \
.show(truncate=False)

""" Using UDF on SQL """
spark.udf.register("convertUDF", convertCase,StringType())
df.createOrReplaceTempView("NAME_TABLE")
spark.sql("select Seqno, convertUDF(Name) as Name from NAME_TABLE") \
.show(truncate=False)

spark.sql("select Seqno, convertUDF(Name) as Name from NAME_TABLE " + \
          "where Name is not null and convertUDF(Name) like '%John%'" ) \
.show(truncate=False)

""" null check """

columns = ["Seqno", "Name"]
data = [("1", "john jones"),
        ("2", "tracey smith"),
        ("3", "amy sanders"),
        ('4',None)]

```

```
df2 = spark.createDataFrame(data=data,schema=columns)
df2.show(truncate=False)
df2.createOrReplaceTempView("NAME_TABLE2")

spark.udf.register("_nullsafeUDF", lambda str: convertCase(str) if not str is None else "" , StringType())

spark.sql("select _nullsafeUDF(Name) from NAME_TABLE2") \
    .show(truncate=False)

spark.sql("select Seqno, _nullsafeUDF(Name) as Name from NAME_TABLE2 " + \
    " where Name is not null and _nullsafeUDF(Name) like '%John%'" ) \
    .show(truncate=False)
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |john jones |
|2     |tracey smith|
|3     |amy sanders |
+-----+-----+
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones  |
|2     |Tracey Smith|
|3     |Amy Sanders |
+-----+-----+
```

```
+-----+-----+-----+
|Seqno|Name      |Cureated Name|
+-----+-----+-----+
|1     |john jones |JOHN JONES   |
|2     |tracey smith|TRACEY SMITH |
|3     |amy sanders |AMY SANDERS  |
+-----+-----+-----+
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones  |
|2     |Tracey Smith|
|3     |Amy Sanders |
+-----+-----+
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones  |
+-----+-----+
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |john jones |
|2     |tracey smith|
|3     |amy sanders |
|4     |NULL       |
+-----+-----+
```

```
+-----+-----+
|_nullsafeUDF(Name)|
+-----+-----+
|John Jones         |
|Tracey Smith       |
|Amy Sanders        |
|                   |
+-----+-----+
```

```
+-----+-----+
|Seqno|Name      |
+-----+-----+
|1     |John Jones  |
+-----+-----+
```